Industrial-Grade Unit Testing and CI/CD for OpenUnderstand

Homework 3 — Advanced Software Testing and Program Analysis

Student ID: 404131058 — Branch: feature/testing-404131058

Table of Contents

1. Introduction

OpenUnderstand is an open-source re-implementation of the SciTools *Understand* Python API for static analysis of Java programs. It models the same core abstractions as the commercial tool — entities, references, lexemes, and the static-analysis relationships between them — and exposes them through a Python facade (`Db`, `Ent`, `Kind`, `Ref`) backed by a peewee/SQLite database.

This report documents an industrial-grade quality-engineering effort on that project: a maintainable unit-testing architecture, a GitHub Actions CI/CD pipeline with enforced quality gates, automated test generation with Pynguin, coverage and mutation analysis, oracle-based validation against the commercial tool, and the discovery and professional reporting of real faults.

The unit chosen for testing in isolation is the public API module `openunderstand/oudb/api.py`, focusing on **entity kinds** and **reference kinds** through the `Kind` class and on the `Ref`/`Ent`/`Db` objects that surface them. Reference kinds (`Java Call`/`Java Callby`, plus the `Define`/`Contain` families) were selected because they directly exercise the two most interesting relationships the assignment asks about: *inverse references* and *parent–child structure*.

2. Testing Architecture

2.1 Isolation strategy

Importing `openunderstand.oudb.api` normally pulls in the entire ANTLR/javalang Java-parsing stack: almost every metric module imports `openunderstand.ounderstand.project`, which in turn builds the generated ANTLR parsers. None of that is needed to test the database-backed api/db layer, and it would make "unit" tests slow, brittle, and non-isolated.

The suite therefore installs lightweight **stub modules** in `sys.modules` before importing the api (`tests/conftest.py`). The unit under test is thus decoupled from its heavy collaborators — a textbook application of the test-double/seam technique. The same shim is reused for Pynguin via a `sitecustomize.py` placed on `PYTHONPATH`.

Each test runs against a fresh **in-memory SQLite** database (peewee `:memory:`), created and dropped per test, so the suite is hermetic, fast, and order-independent, and never touches a developer's real `.oudb` files.

2.2 Layout

```
tests/
├── conftest.py            # isolation bootstrap + shared fixtures
├── test_kind.py           # entity & reference kinds, inverse references
├── test_ref.py            # reference objects
├── test_entity_and_db.py  # parent-child, unknown entities, db queries
├── test_misc.py           # remaining in-scope accessors / dunders
├── _pynguin_support/      # sitecustomize shim for Pynguin
└── generated/             # curated Pynguin output
```

Fixtures provide a realistic slice of the Java kind table (with inverse `_inv` wiring identical to `oudb/fill.py`), a `File → Class → Method → Parameter` entity tree, a `Java Call` reference, and an `api.Db` bound to the in-memory database. Factory fixtures

(`make_kind`/`make_ent`/`make_ref`) wrap ORM rows as api dataclasses exactly as the production code does.

3. Manual Unit Testing

Handcrafted tests cover the eight behaviours required by Part 3:

#	Requirement	Where
1	Handcrafted unit tests	all `tests/test_*.py`
2	Normal behaviour	name/longname/check/list_*/accessors
3	Malformed input	reference to a non-existent entity id (`test_ref`)
4	Edge cases	empty filter string, no-match fallback (`test_kind`)
5	Unresolved/unknown entities	`ent_from_id(unknown)` → `None`; unknown kind filter → empty
6	Inverse references	`Kind.inv()` (entity-kind raises; reference-kind via xfail)
7	Parent–child relationships	full `parent()` chain walk (`test_entity_and_db`)
8	Multi-pass analysis	parser-stage concern; documented + skipped placeholder

Requirement 8 (multi-pass analysis) is a property of the parser/listener stage, which is intentionally stubbed for isolated unit testing; it is addressed instead by oracle-based/differential testing (Section 6) and marked with an explicit `skip` so the intent is recorded.

Two behaviours are encoded as `xfail` tests because they expose genuine bugs (Section 7); marking them as expected failures keeps the pipeline green while still documenting the defects.

4. Automated Test Generation (Pynguin)

Pynguin (DYNAMOSA, fixed seed for reproducibility) targets `openunderstand.oudb.api`. Because Pynguin pins old transitive dependencies, it runs in its own virtual environment; the `scripts/run_pynguin.*` wrappers set `PYNGUIN_DANGER_AWARE` and put the isolation shim on `PYTHONPATH` so generation focuses on the api/db layer rather than the parser.

Generated tests are treated as raw material, not deliverables. The intended workflow (documented in `tests/generated/README.md`) is: **evaluate** the coverage delta, **remove** flaky/meaningless cases (identity, ordering, timestamps, random junk assertions), **refactor** for readability, and **integrate** the survivors under `tests/` so CI runs them automatically.

4.1 Empirical result: Pynguin could not generate tests for this module

Running Pynguin against `openunderstand.oudb.api` was attempted in a dedicated `.venv-pynguin` (Python 3.12) via `scripts/run_pynguin.ps1`. Generation **failed before producing any tests**, in two distinct stages:

1. **`RecursionError: maximum recursion depth exceeded`** — Pynguin snapshots the module state with `dill`, which recursively pickles every object reachable from the target module. `api.py` reaches the peewee ORM classes (`EntityModel`, `KindModel`, `ReferenceModel`), whose metaclass/field graphs are deeply self-referential, overflowing CPython's default 1000-frame limit. *Mitigation attempted:* raising the interpreter recursion limit to 30 000 in the Pynguin start-up shim (`tests/_pynguin_support/sitecustomize.py`). This got past the serialization stage.
2. **`NameError: name 'IterableClassWatcher' is not defined`** — an *internal* Pynguin symbol referenced by its runtime instrumentation/type-tracing layer is undefined in the installed release. Because the missing name belongs to Pynguin itself (not to the module under test), it is a tool-side regression that no configuration of the target can work around.

Analysis. Pynguin's DYNAMOSA engine relies on (a) `dill`-serialising module state and (b) instrumenting the return values of the unit under test to seed its type system. Both mechanisms are hostile to a database-facade module: the ORM object graph is unserialisable in practice, and the API returns ORM-wrapped dataclasses that trigger Pynguin's `IterableClass` watcher path — exactly where the internal `NameError` fires. This is a **known class of limitation** for search-based generators on framework/ORM-bound code, and is reported here as a genuine engineering finding rather than papered over with fabricated numbers.

Consequence for the suite. Automated generation added no tests, so coverage of the api/db layer rests entirely on the handcrafted suite (Part 3) — which already reaches **97.13 % line / 93.18 % branch**, comfortably above target. The `scripts/run_pynguin.*` wrappers, the isolation shim, and the curate→evaluate→refactor→integrate workflow in `tests/generated/README.md` are retained so the pipeline is ready to absorb generated tests on a Pynguin release where the instrumentation bug is fixed, or against a pure-Python (ORM-free) module.

5. Coverage and Mutation Analysis

5.1 Coverage

Coverage is measured with branch tracking on the modules under test (`openunderstand/oudb/api.py` and `models.py`). The large `metric()`/`metrics()` engine and the file/git/info-browser integration methods are explicitly marked `# pragma: no cover`: they require the ANTLR parser and a populated `.udb` database and are therefore out of *unit*-test scope (they belong to integration/oracle testing). This scoping is deliberate and documented, and keeps the coverage figure meaningful for the layer actually under test.

Quality gates enforced by CI: **line ≥ 80 %**, **branch ≥ 70 %**.

Empirical result (Python 3.12.8, `pytest`, branch coverage on `api.py` + `models.py`):

Metric	Result	Gate	Pass?
Tests passed	68 passed, 1 skipped, 2 xfailed (71 collected)	all green	Pass
Line coverage	97.13 %	≥ 80 %	Pass
Branch coverage	93.18 % (41 / 44)	≥ 70 %	Pass

Per-module: `api.py` 98.82 % line coverage (294 stmts, 1 missed), `models.py` 84.44 % (45 stmts, 7 missed). The two `xfail` tests document real faults (Section 7); the one skip is the parser-stage multi-pass concern.

5.2 Mutation testing

Mutation testing was configured for both mutmut (`setup.cfg`) and Cosmic Ray (`cosmic-ray.toml`). **mutmut has no native Windows support** (mutmut issue #397), so on the Windows development machine the **Cosmic Ray** engine was used instead — this is exactly why the repository ships a cross-platform fallback configuration. The run mutates `openunderstand/oudb/api.py` and executes the full unit suite (`python -m pytest tests -x -q`) against each mutant.

Empirical result (Cosmic Ray, whole-module run over `api.py`):

Quantity	Value
Total mutants	839
Killed	104
Survived	735
Mutation score (killed / total)	**12.40 %**
Survival rate	87.60 %

Interpreting the score

The headline 12.40 % is a **whole-file** score and materially understates the effectiveness of the suite on the code it actually targets. Cosmic Ray mutates *every* statement in the ~1 600-line `api.py`, including the large `metric()`/`metrics()` engine and the file/git/info-browser integration methods. Those regions are **deliberately outside unit-test scope** — they require the ANTLR parser and a populated `.udb` database and are marked `# pragma: no cover` in the coverage configuration (Section 5.1). Mutants planted in code the unit suite never executes **cannot** be killed by that suite, so they inflate the surviving count.

The surviving mutants cluster into three categories:

1. **Out-of-scope integration code (the large majority).** Hundreds of `ReplaceComparisonOperator` and `NumberReplacer` mutants land in the metric/file/browser methods that unit tests intentionally do not exercise.
2. **Equivalent mutants.** Many `ReplaceComparisonOperator_Eq_Is` / `Eq_IsNot` survivors are semantically equivalent — for comparisons against `None` or interned singletons, `==` and `is` behave identically, so no test can distinguish them. These are unkillable in principle and are correctly *not* counted as test weaknesses.
3. **Genuinely killable survivors in in-scope code (a small minority).** A handful of mutants in the `Kind`/`Ref`/`Ent`/`Db` accessors that the suite *does* cover but does not assert tightly enough (e.g. a boundary comparison whose exact operator the assertions don't pin down).

Improving the tests accordingly

The correct methodological fix — and the way to obtain a *representative* score — is to re-scope the mutation target to the same surface as the coverage configuration (the unit-tested `Kind`/`Ref`/`Ent`/`Db` methods) rather than the whole facade. For the third category above, tightening assertions on the exact returned values and boundary conditions kills the remaining in-scope survivors; categories 1 and 2 are addressed by scoping and by marking equivalent mutants, not by adding tests. Chasing the bonus **> 85 % mutation score** target would require either that re-scoping or unit-testing the integration engine (which belongs to oracle/integration testing, Section 6), and was out of scope for this submission.

6. Oracle-Based Validation

Status: not executed. Oracle-based differential testing against the commercial SciTools *Understand* tool was **not performed** for this submission, because a licensed Understand installation was not available on the development machine. No oracle-derived result is claimed anywhere in this report. Differential testing against Understand is listed as a *Bonus Challenge* in the assignment.

Design that was prepared for it. The intended approach uses Understand as the oracle: for a small Java sample, the same queries would be issued through Understand's Python

API and through OpenUnderstand, and the resulting entity/reference graphs compared. That would validate semantics pure unit tests cannot reach — e.g. that `Java Call`/`Java Callby` are produced with the correct scope/ent orientation, and that parent–child containment matches Understand's. Because Understand is licensed and platform-specific, such a stage would run locally rather than in CI. The isolation seams built for the unit suite (`tests/conftest.py`, `tests/_pynguin_support/sitecustomize.py`) are structured so a differential harness can be dropped in where a license is present.

7. Fault Discovery

Code review while building the suite surfaced four faults. The two confirmed, test-backed ones are reproduced by `xfail` tests in the suite and written up as complete, ready-to-file GitHub issue reports in `docs/FAULTS.md` (see that file for the issue links once filed against https://github.com/m-zakeri/OpenUnderstand/issues).

1. **`Kind.inv()` always raises `TypeError`.** It calls `inverse.__data__.get("__data__")`, which is `None` (every other call site correctly uses `.__dict__.get("__data__")`), so the inverse of *any* reference kind is unreachable. One-line fix proposed.
2. **`Db.lookup(name)` without a kind filter always returns `[]`** because the final `re.search` builds the literal pattern `java\s+None`. Guarded-fix proposed.
3. **`Ent.refs()` crashes when called with no arguments** (`None.split(",")`) despite the argument being documented as optional; it also contains leftover `print` debug statements.
4. **`Db.ents()` multi-token filtering is contradictory** (ANDs per-token `_kind IN (...)` subqueries that can never be simultaneously satisfied), already flagged in-code with a `TODO`.

Each issue includes environment, reproduction steps, expected vs. observed behaviour, the failing test, and a suggested fix for an optional pull request.

8. Lessons Learned

- **Isolation is a design activity.** The single highest-leverage decision was stubbing the parser stack so the api/db layer could be tested as a true unit; it made the suite fast, deterministic, and runnable anywhere — including in CI on three Python versions.
- **Coverage scope must be honest and explicit.** Rather than chase an unreachable 80 % over a 1,600-line facade that embeds an integration-only metric engine, the engine was explicitly excluded from *unit* scope and earmarked for oracle testing. Gaming coverage and declaring scope look similar but are not: the difference is documentation and intent.

- **Tests are excellent bug detectors.** Simply trying to assert the *correct* behaviour of `Kind.inv()` and `Db.lookup()` immediately exposed real defects; `xfail` lets a suite document bugs without going red.
- **Tooling reproducibility matters.** Pinned dev requirements, pip caching, `PYTHONHASHSEED=0`, and a Dockerfile make the pipeline reproducible; Pynguin's dependency pins justified a separate environment.
- **Automated generation is not universally applicable.** Pynguin could not run on the ORM-backed API module at all — it failed first on `dill` recursion and then on an internal instrumentation bug (`IterableClassWatcher`). Knowing when a tool's assumptions (serialisable state, simple return types) are violated, and documenting that honestly, is itself part of the engineering job; the manual suite already exceeded every coverage gate without it.

9. Deliverables and Where to Find Them

Deliverable	Location in repository
Unit tests (handcrafted)	`tests/` (`test_kind.py`, `test_ref.py`, `test_entity_and_db.py`, `test_misc.py`)
Test fixtures / isolation	`tests/conftest.py`, `tests/_pynguin_support/sitecustomize.py`
CI/CD pipeline	`.github/workflows/ci.yml`
Coverage reports	`coverage.xml`, `coverage.json`, `htmlcov/index.html`, `reports/junit.xml`
Mutation config	`cosmic-ray.toml` (Cosmic Ray), `setup.cfg` (mutmut)
Pynguin scripts	`scripts/run_pynguin.sh` / `.ps1`
Mutation scripts	`scripts/run_mutation.sh` / `.ps1`
Fault reports (GitHub issues)	`docs/FAULTS.md`
Testing / reproduce guide	`docs/TESTING.md`
This report	`docs/REPORT.md` (source), `docs/OpenUnderstand-HW3-Final-Report.docx`, `docs/OpenUnderstand-HW3-Final-Report.pdf`
Reproducible container	`Dockerfile`, `.dockerignore`

Appendix A — How to reproduce

Any of Python 3.10, 3.11 or 3.12 works (CI tests all three). Every command is a single line, so it can be pasted into PowerShell, `cmd` or a POSIX shell without edits. Full step-by-step instructions are in `docs/TESTING.md`.

Windows (PowerShell), from the repo root:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
ruff check tests
pytest tests --cov=openunderstand.oudb.api --cov=openunderstand.oudb.models --cov-branch --cov-report=term-missing --cov-report=html --cov-report=json
.\scripts\run_mutation.ps1
```

Steps 4–5 above are the lint gate (Parts 1–2) and the unit tests plus coverage (Parts 3 & 5); step 6 is mutation testing (Part 5, Cosmic Ray — mutmut has no native Windows support).

Linux/macOS: identical, except `source .venv/bin/activate` and `./scripts/run_mutation.sh` (mutmut).

Pynguin (Part 4) needs its own environment because it pins older dependencies:

```
python -m venv .venv-pynguin
.\.venv-pynguin\Scripts\Activate.ps1
pip install "pynguin>=0.43.0" -r requirements.txt
.\scripts\run_pynguin.ps1
```

See `docs/TESTING.md` for the full guide, including the CI job breakdown and the Docker workflow.